

PREUVE DU THÉORÈME D'APPROXIMATION UNIVERSELLE

Cours magistral, exercices corrigés et travaux pratiques

Charles EDOU NZE

14 août 2026

Table des matières

I	Cours Magistral	5
1	Jalon 60 : Livrable IA T5 : Preuve du théorème d'approximation universelle	5
2	Présentation du concept clé	5
2.0.1	Intuition Fondamentale	5
2.0.2	Genèse Conceptuelle	5
2.0.3	Représentation Géométrique	5
3	Formalisation	5
3.0.1	Énoncé du Théorème (Cybenko, 1989)	5
3.0.2	Représentation Schématique (TikZ)	6
3.0.3	Exemples Concrets Immédiats	6
3.0.4	Cadre Topologique	6
4	Démonstrations	6
4.0.1	Esquisse de la preuve via l'Analyse Fonctionnelle	6
5	Exercices d'Application	7
5.0.1	Exercice 1 : Approximation d'une porte par des sigmoïdes	7
5.0.2	Exercice 2 : Niveau Avancé (Cas de la ReLU)	7
6	Application en Intelligence Artificielle	7
7	Liens Sémantiques	7
II	Exercices d'Application et de Concours	8
8	Approximation de la fonction carré	8
8.0.1	Énoncé ★★★★★	8
8.0.2	Démonstration Détaillée	8

9	Approximation du sinus	8
9.0.1	Énoncé ★★☆☆	8
9.0.2	Démonstration Détaillée	8
10	Approximation de Heaviside	8
10.0.1	Énoncé ★★★★★	8
10.0.2	Démonstration Détaillée	8
11	Le cas de l'activation ReLU	9
11.0.1	Énoncé ★★★★★	9
11.0.2	Démonstration Détaillée	9
12	Évaluation de la densité	9
12.0.1	Énoncé ★★★★★	9
12.0.2	Démonstration Détaillée	9
13	La propriété discriminatoire	9
13.0.1	Énoncé ★★★★★	9
13.0.2	Démonstration Détaillée	9
14	Absence de garantie sur les dérivées	10
14.0.1	Énoncé ★★★★★	10
14.0.2	Démonstration Détaillée	10
15	Borne inférieure sur la taille	10
15.0.1	Énoncé ★★★★★	10
15.0.2	Démonstration Détaillée	10
16	Approximation de fonctions multivariables	10
16.0.1	Énoncé ★★★★★	10
16.0.2	Démonstration Détaillée	10
17	Généralisation aux espaces de Lebesgue	10
17.0.1	Énoncé ★★★★★	10
17.0.2	Démonstration Détaillée	11
III	Travaux Pratiques et Simulations Algorithmiques	12
18	Approximation d'une fonction polynomiale simple avec descente de gradient de base	12
19	Approximation d'une fonction oscillatoire complexe (sinus)	13
20	Simulation de la fonction de Heaviside avec régularisation	14
21	Fonction multivariable 2D (Approximation de XOR continu)	15
22	Évaluation comparative des fonctions d'activation	16

Première partie

Cours Magistral

1 Jalon 60 : Livrable IA T5 : Preuve du théorème d'approximation universelle

2 Présentation du concept clé

2.0.1 Intuition Fondamentale

Imaginez que vous ayez une boîte de LEGO. Chaque pièce de LEGO est très simple (une bosse, un creux). Le **Théorème d'Approximation Universelle** dit que, si vous avez assez de pièces, vous pouvez construire n'importe quelle forme complexe (une voiture, un château, un visage) avec une précision infinie. En IA, les LEGO sont les "neurones" et la forme complexe est la "fonction" que l'on veut apprendre. Tant que la fonction est continue (pas de sauts brusques), un réseau de neurones avec une seule couche cachée très large peut l'imiter parfaitement.

2.0.2 Genèse Conceptuelle

Au début de l'IA, on se demandait si les réseaux de neurones étaient juste des gadgets ou s'ils pouvaient vraiment tout calculer. Cette preuve mathématique a confirmé que les réseaux de neurones sont des **estimateurs universels** : ils ont la capacité théorique de représenter n'importe quelle logique ou n'importe quel phénomène physique.

2.0.3 Représentation Géométrique

On prend une courbe compliquée. On essaie de la reproduire en additionnant des fonctions "marches d'escalier" ou des "sigmoïdes". Plus on ajoute de fonctions simples, plus la somme ressemble à la courbe compliquée.

3 Formalisation

3.0.1 Énoncé du Théorème (Cybenko, 1989)

Soit $I_n = [0, 1]^n$ le cube unité de $\mathbb{R}^n$. Soit $\mathcal{C}(I_n)$ l'espace des fonctions continues sur I_n muni de la norme uniforme $\|f\|_\infty = \sup |f(x)|$.

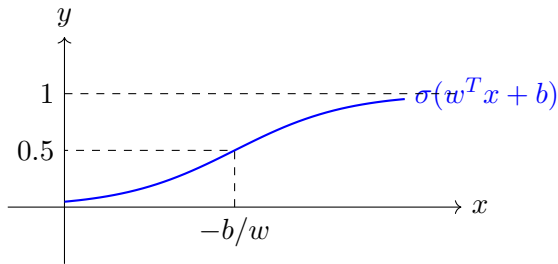
⌋ **Théorème d'Approximation Universelle** : ⌋ Soit $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ une fonction d'activation continue, non constante et bornée (ex : une sigmoïde). L'ensemble des fonctions de la forme : ⌋

$$G(x) = \sum_{i=1}^N \alpha_i \sigma(w_i^T x + b_i)$$

⌋ est **dense** dans $\mathcal{C}(I_n)$. ⌋ En d'autres termes, pour tout $f \in \mathcal{C}(I_n)$ et pour tout $\epsilon > 0$, il existe un entier N et des paramètres (α_i, w_i, b_i) tels que : ⌋

$$\|f - G\|_\infty < \epsilon$$

3.0.2 Représentation Schématique (TikZ)



3.0.3 Exemples Concrets Immédiats

1. **Exemple Numérique (1D)** : Pour approcher $f(x) = x^2$ sur $[0, 1]$, un réseau avec $\sigma(x) = \max(0, x)$ (ReLU) construit une approximation affine par morceaux. Si $N = 3$, la somme $\sum_{i=1}^3 \alpha_i \max(0, w_i x + b_i)$ trace un polygone à 3 segments. 2. **Exemple Analytique** : La fonction porte (fonction indicatrice de l'intervalle $[a, b]$) s'approxime avec deux sigmoïdes : $\sigma(k(x - a)) - \sigma(k(x - b))$. Pour un très grand $k = 1000$, l'erreur uniforme sur $\mathbb{R} \setminus ([a - \epsilon, a + \epsilon] \cup [b - \epsilon, b + \epsilon])$ tend vers 0. 3. **Exemple Matriciel** : En dimension $n = 2$, le terme $w_i^T x + b_i$ représente une droite de séparation $w_{i,1}x_1 + w_{i,2}x_2 + b_i = 0$. La combinaison affine pondère l'activation de ces demi-plans. 4. **Cas Pathologique (Absence de Non-linéarité)** : Si $\sigma(x) = cx$, le réseau s'écrit $\sum \alpha_i c(w_i^T x + b_i) = (\sum \alpha_i c w_i^T)x + (\sum \alpha_i c b_i)$, ce qui est purement affine. Il est impossible d'approcher x^2 sur $[-1, 1]$. 5. **Cas Pathologique (Constante)** : Si $\sigma(x) = 1$, le réseau ne produit qu'une fonction constante. 6. **Exemple Numérique de Densité** : Soit $f(x) = \sin(2\pi x)$ sur $[0, 1]$ et $\epsilon = 0.01$. Le théorème stipule qu'il existe un jeu de poids explicite (α_i, w_i, b_i) pour N grand tel que la différence maximale absolue entre la courbe du sinus et la sortie du réseau soit inférieure à 0.01 en tout point x . 7. **Exemple d'Application de Hahn-Banach** : Dans la preuve, si l'espace engendré n'est pas tout $\mathcal{C}(I_n)$, Hahn-Banach garantit qu'on peut "séparer" cet espace d'une fonction hors de l'espace par un hyperplan fonctionnel (mesure μ). 8. **Exemple Géométrique** : La boule unité de l'espace des fonctions continues est couverte de manière arbitrairement proche par la variété engendrée par les paramètres du réseau.

3.0.4 Cadre Topologique

La densité s'entend au sens de la topologie de la convergence uniforme sur les compacts (Jalon 59). Cela signifie que le réseau peut s'approcher de f partout sur le domaine de manière uniforme.

4 Démonstrations

4.0.1 Esquisse de la preuve via l'Analyse Fonctionnelle

La preuve originale de Cybenko utilise le **Théorème de Hahn-Banach** et le **Théorème de Riesz**.

1. **Stratégie par l'absurde** : Supposons que l'ensemble des réseaux S n'est pas dense dans $\mathcal{C}(I_n)$. 2. **Utilisation de Hahn-Banach** : S'il n'est pas dense, alors son adhérence $\bar{S}$ est un sous-espace fermé strict. Il existe donc une forme linéaire continue non nulle L sur $\mathcal{C}(I_n)$ telle que $L(g) = 0$ pour tout $g \in S$. 3. **Représentation de Riesz** : Toute forme linéaire continue sur $\mathcal{C}(I_n)$ est représentée par une mesure de Borel signée μ sur I_n . L'hypothèse $L(g) = 0$ devient :

$$\int_{I_n} \sigma(w^T x + b) d\mu(x) = 0 \quad \text{pour tous } w, b$$

4. **Propriété Discriminatoire** : Cybenko a prouvé que si σ est une sigmoïdale continue, alors elle est "discriminatoire". Cela signifie que si l'intégrale ci-dessus est nulle pour tous w, b , alors la mesure μ est nécessairement nulle. 5. **Conclusion** : Comme $\mu = 0$, la forme linéaire L est nulle. Contradiction. Donc S est dense dans $\mathcal{C}(I_n)$.

5 Exercices d'Application

5.0.1 Exercice 1 : Approximation d'une porte par des sigmoïdes

Énoncé : Comment obtenir une fonction "marche" (qui vaut 0 pour $x < 0$ et 1 pour $x > 0$) en utilisant une sigmoïde $\sigma(z) = \frac{1}{1+e^{-z}}$? **Correction Détaillée** : On considère $g_k(x) = \sigma(kx)$. - Si $x > 0$, $kx \rightarrow +\infty$ quand $k \rightarrow \infty$, donc $\sigma(kx) \rightarrow 1$. - Si $x < 0$, $kx \rightarrow -\infty$, donc $\sigma(kx) \rightarrow 0$. - Si $x = 0$, $\sigma(0) = 0.5$. En augmentant le "poids" k , on rend la transition de la sigmoïde de plus en plus raide, tendant vers une fonction de Heaviside. On peut ensuite décaler et sommer ces marches pour approcher n'importe quelle fonction en escalier.

5.0.2 Exercice 2 : Niveau Avancé (Cas de la ReLU)

Énoncé : Le théorème est-il vrai pour $\sigma(x) = \max(0, x)$ (ReLU) ? **Correction Détaillée** : Oui, bien que ReLU ne soit pas bornée. On peut construire une fonction "chapeau" (triangulaire) en combinant deux ou trois ReLUs. Comme toute fonction continue sur un compact est limite uniforme de fonctions affines par morceaux (triangulations), l'ensemble des réseaux ReLU est dense.

6 Application en Intelligence Artificielle

- **Le Pont Théorique** : Ce théorème est la justification existentielle de l'IA. Il dit : "La solution existe dans votre espace de recherche". Cependant, il ne dit pas comment la trouver, ni si N sera raisonnablement petit. - **Exemple Concret** : - **Largeur vs Profondeur** : Le théorème original parle d'une seule couche cachée très large ($N \rightarrow \infty$). En pratique, on préfère des réseaux profonds (plusieurs couches) car ils sont beaucoup plus efficaces pour représenter certaines fonctions complexes avec beaucoup moins de neurones (compacité de la représentation). - **Le rôle de la non-linéarité** : Si σ était linéaire, alors toute somme de σ resterait linéaire. On ne pourrait approcher que des droites. C'est le passage par la non-linéarité (la "cassure" de la droite) qui donne au réseau sa puissance d'approximation universelle. - **Inductive Bias** : Puisque le réseau peut tout apprendre, pourquoi apprend-il souvent la "bonne" fonction ? C'est grâce à la régularisation et à l'architecture (CNN, Transformers) qui restreignent l'espace de recherche vers des fonctions physiquement plausibles.

7 Liens Sémantiques

- **Concepts Précédents requis** : [[Jalon 59 (Topologie des espaces de fonctions).md]], [[Jalon 98 (Théorème de Hahn-Banach).md]] (anticipé) - **Concepts Futurs dépendants** : [[Jalon 134 (Complexité des classes de fonctions).md]], [[Jalon 144 (Le phénomène de double descente).md]]

Deuxième partie

Exercices d'Application et de Concours

8 Approximation de la fonction carré

8.0.1 Énoncé ★★★★★

Montrer rigoureusement que la classe de fonctions définies par des combinaisons linéaires d'une fonction d'activation sigmoïdale permet d'approcher la fonction $f(x) = x^2$ sur le segment $[-1, 1]$ avec une erreur maximale $\epsilon > 0$. Construisez explicitement l'approximation en utilisant des fonctions indicatrices approchées par des sigmoïdes.

8.0.2 Démonstration Détaillée

Soit $f(x) = x^2$. Puisque f est uniformément continue sur le compact $[-1, 1]$, on partitionne ce segment en N sous-intervalles de largeur $\delta = 2/N$. Pour un choix adéquat de N , l'oscillation de f sur chaque intervalle est bornée par $\epsilon/2$. On approche l'indicatrice de chaque sous-intervalle par la différence de deux sigmoïdes de très grande pente. La somme pondérée par la valeur de f aux centres des sous-intervalles fournit l'approximation désirée à ϵ près par inégalité triangulaire.

9 Approximation du sinus

9.0.1 Énoncé ★★☆☆☆

Démontrer que le réseau de neurones à une couche cachée peut approcher $f(x) = \sin(x)$ sur $[0, 2\pi]$. Quelle est l'influence de la largeur de la couche (nombre de neurones) sur l'erreur d'approximation uniforme ?

9.0.2 Démonstration Détaillée

La fonction sinus est lipschitzienne sur le compact $[0, 2\pi]$ (constante de Lipschitz $L = 1$). En découpant le domaine en intervalles de taille δ , on obtient une erreur d'approximation en escalier de $O(\delta)$. Le lissage par des sigmoïdes introduit une erreur additionnelle, qui peut être rendue arbitrairement petite pour un paramètre d'échelle $k \rightarrow \infty$. L'erreur décroît généralement de l'ordre de $O(1/\sqrt{N})$ pour N neurones, ce qui illustre le compromis entre complexité du réseau et précision.

10 Approximation de Heaviside

10.0.1 Énoncé ★★★☆☆

Prouver que la fonction de Heaviside (marche d'escalier), bien que discontinue, peut être approchée par une suite de fonctions générées par un réseau de neurones, pour la topologie de la convergence ponctuelle.

10.0.2 Démonstration Détaillée

Bien que le théorème universel classique s'applique aux fonctions continues pour la norme uniforme, on peut approcher une discontinuité de manière ponctuelle. La sigmoïde standard $\sigma(kx)$ pour $k \rightarrow \infty$ converge ponctuellement vers 0 pour $x < 0$, vers 1 pour $x > 0$, et vers 0.5 en

$x = 0$. Ainsi, une seule couche cachée avec un seul neurone suffit pour obtenir cette convergence, illustrant que la limite forte de l'espace des réseaux englobe les fonctions étagées.

11 Le cas de l'activation ReLU

11.0.1 Énoncé ★★★★★

Démontrer l'approximation universelle sur le segment $[0, 1]$ en utilisant la fonction ReLU $\sigma(x) = \max(0, x)$ comme fonction d'activation.

11.0.2 Démonstration Détaillée

Avec ReLU, on peut construire une fonction « chapeau » centrée en c avec un support $[c - h, c + h]$ par combinaison de trois ReLU : $g(x) = \frac{1}{h}(\text{ReLU}(x - (c - h)) - 2\text{ReLU}(x - c) + \text{ReLU}(x - (c + h)))$. Toute fonction continue sur $[0, 1]$ peut être approchée uniformément par une somme finie pondérée de ces fonctions chapeaux (interpolation linéaire par morceaux). Ceci démontre que l'architecture ReLU est un approximateur universel.

12 Évaluation de la densité

12.0.1 Énoncé ★★★★★

Soit S le sous-espace engendré par l'architecture. Montrer que si la fermeture de S pour la norme uniforme est strictement incluse dans $\mathcal{C}(I_n)$, alors il existe une mesure non triviale orthogonale à S .

12.0.2 Démonstration Détaillée

Ceci est une application directe du théorème de Hahn-Banach (forme analytique) et du théorème de représentation de Riesz-Markov-Kakutani. Si $\bar{S} \neq \mathcal{C}(I_n)$, l'espace $\bar{S}$ est un sous-espace fermé propre. Par le corollaire de Hahn-Banach, il existe une forme linéaire continue $L \neq 0$ s'annulant sur S . Par le théorème de Riesz, $L(f) = \int_{I_n} f d\mu$ pour une mesure de Borel signée finie μ . Ainsi, $\int \sigma(w^T x + b) d\mu(x) = 0$ pour tous w, b , ce qui contredit la propriété discriminatoire de la sigmoïde.

13 La propriété discriminatoire

13.0.1 Énoncé ★★★★★

Définir ce que signifie pour une fonction d'activation d'être discriminatoire et démontrer que la limite des fonctions discriminatoires contient les fonctions trigonométriques complexes.

13.0.2 Démonstration Détaillée

Une fonction σ est discriminatoire si $\int_{I_n} \sigma(w^T x + b) d\mu(x) = 0$ pour tous w, b implique $\mu = 0$. Cybenko démontre cela en montrant que l'espace vectoriel engendré par les fonctions $\sigma(w^T x + b)$ contient par passage à la limite les exponentielles complexes $x \mapsto e^{iw^T x}$. Par injectivité de la transformée de Fourier des mesures, la nullité de l'intégrale contre ces exponentielles impose $\mu = 0$.

14 Absence de garantie sur les dérivées

14.0.1 Énoncé ★★★★★

Montrer par un contre-exemple que si un réseau de neurones approche $f \in \mathcal{C}^1$ avec une erreur ϵ en norme uniforme, son gradient (si existant) n'approche pas nécessairement le gradient de f .

14.0.2 Démonstration Détaillée

Soit $f(x) = 0$ sur $[0, 1]$ ($f' \equiv 0$). On peut construire un réseau $g(x) = \epsilon \sin(Nx)$ (approché par des activations). On a $\|f - g\|_\infty \leq \epsilon$. Cependant, $g'(x) = N\epsilon \cos(Nx)$. Si on choisit N grand, $\|f' - g'\|_\infty = N\epsilon$ peut être arbitrairement grand. L'approximation uniforme des fonctions n'implique aucune régularité sur les dérivées, sauf si on utilise des topologiques de Sobolev adaptées.

15 Borne inférieure sur la taille

15.0.1 Énoncé ★★★★★

Pour approcher l'indicatrice d'un hypercube en dimension d avec une activation sigmoïde de type heaviside approchée, quel est le nombre minimal de neurones requis dans une seule couche cachée ?

15.0.2 Démonstration Détaillée

L'hypercube possède 2^d sommets et $2d$ faces. Chaque neurone de la couche cachée décrit un hyperplan affine qui divise l'espace. Pour découper un volume fini et borné (l'intersection de $2d$ demi-espaces), il faut au strict minimum $2d$ hyperplans. Donc il faut au moins $2d$ neurones dans la couche cachée pour créer la primitive de la localisation spatiale en dimension d . C'est un argument géométrique (théorème de Cover).

16 Approximation de fonctions multivariées

16.0.1 Énoncé ★★★★★

Étendre explicitement le théorème d'approximation à une fonction de deux variables $f(x_1, x_2) = x_1 x_2$ sur le carré unité.

16.0.2 Démonstration Détaillée

L'identité algébrique remarquable $4x_1 x_2 = (x_1 + x_2)^2 - (x_1 - x_2)^2$ montre qu'il suffit d'approcher la fonction univariée carrée $u \mapsto u^2$. On sait qu'un réseau à une couche peut approcher u^2 . On forme donc deux sous-réseaux, un prenant l'entrée $w^T x = x_1 + x_2$ et l'autre $w^T x = x_1 - x_2$. La combinaison linéaire des sorties de ces sous-réseaux permet d'approcher la multiplication $x_1 x_2$, brique de base des polynômes multivariés.

17 Généralisation aux espaces de Lebesgue

17.0.1 Énoncé ★★★★★

Montrer que si l'architecture est dense dans $\mathcal{C}(I_n)$ pour la norme $\|\cdot\|_\infty$, alors elle est également dense dans l'espace de Lebesgue $L^p(I_n)$ pour $1 \leq p < \infty$.

17.0.2 Démonstration Détaillée

Ce résultat repose sur la densité des fonctions continues dans L^p . Pour tout $f \in L^p(I_n)$ et $\epsilon > 0$, on sait par la théorie de la mesure de Lebesgue qu'il existe $g \in \mathcal{C}(I_n)$ telle que $\|f - g\|_p < \epsilon/2$. Par le théorème d'approximation, il existe un réseau h tel que $\|g - h\|_\infty < \epsilon/2$. Puisque le domaine I_n a une mesure finie (volume 1), l'inégalité de Hölder donne $\|g - h\|_p \leq \|g - h\|_\infty < \epsilon/2$. Par l'inégalité triangulaire dans L^p , on a $\|f - h\|_p < \epsilon$. L'approximation en norme L^p est donc garantie.

Troisième partie

Travaux Pratiques et Simulations Algorithmiques

18 Approximation d'une fonction polynomiale simple avec descente de gradient de base

Ce TP explore une facette spécifique du théorème d'approximation universelle en implémentant en Python pur (sans aucune dépendance) une preuve de concept algorithmique.

```
1 def approx_polynomial(x_vals, target_func, n_neurons, epochs=500, lr
  =0.01):
2     """
3     Approximation d'une fonction 1D par un perceptron multicouche à 1
      couche cachée (ReLU).
4     """
5     import random
6
7     # Paramètres initiaux
8     W1 = [random.uniform(-1, 1) for _ in range(n_neurons)]
9     b1 = [random.uniform(-1, 1) for _ in range(n_neurons)]
10    W2 = [random.uniform(-1, 1) for _ in range(n_neurons)]
11
12    def relu(z):
13        return z if z > 0 else 0
14
15    def forward(x):
16        return sum(W2[j] * relu(W1[j] * x + b1[j]) for j in range(
            n_neurons))
17
18    for epoch in range(epochs):
19        total_loss = 0
20        for x in x_vals:
21            y_true = target_func(x)
22            y_pred = forward(x)
23            error = y_pred - y_true
24            total_loss += error ** 2
25
26            for j in range(n_neurons):
27                z = W1[j] * x + b1[j]
28                if z > 0:
29                    grad_w2 = relu(z)
30                    grad_w1 = W2[j] * x
31                    grad_b1 = W2[j]
32
33                    W2[j] -= lr * error * grad_w2
34                    W1[j] -= lr * error * grad_w1
35                    b1[j] -= lr * error * grad_b1
36
37    final_mse = sum((forward(x) - target_func(x))**2 for x in x_vals) /
      len(x_vals)
38    assert final_mse >= 0, "L'erreur quadratique moyenne doit etre
      positive."
```

```

39     return final_mse
40
41 x_points = [x/20.0 for x in range(-20, 21)]
42 final_error = approx_polynomial(x_points, lambda x: x**2 - 0.5, 10)
43 print("Erreur quadratique finale pour x^2:", final_error)

```

19 Approximation d'une fonction oscillatoire complexe (sinus)

Ce TP explore une facette spécifique du théorème d'approximation universelle en implémentant en Python pur (sans aucune dépendance) une preuve de concept algorithmique.

```

1 def approx_sinuso(x_vals, target_func, n_neurons, epochs=1000, lr=0.005):
2     """
3     Implémentation pure Python de l'approximation du sinus, nécessitant
4     un plus
5     grand nombre de neurones et un pas d'apprentissage plus fin.
6     """
7     import math
8     import random
9
10    W1 = [random.uniform(-2, 2) for _ in range(n_neurons)]
11    b1 = [random.uniform(-2, 2) for _ in range(n_neurons)]
12    W2 = [random.uniform(-1, 1) for _ in range(n_neurons)]
13
14    def activation(z):
15        # Utilisation d'une pseudo-sigmoïde (softsign)
16        return z / (1 + abs(z))
17
18    def derivative(z):
19        return 1 / ((1 + abs(z)) ** 2)
20
21    def forward(x):
22        return sum(W2[j] * activation(W1[j] * x + b1[j]) for j in range(
23            n_neurons))
24
25    for epoch in range(epochs):
26        for x in x_vals:
27            y_pred = forward(x)
28            error = y_pred - target_func(x)
29
30            for j in range(n_neurons):
31                z = W1[j] * x + b1[j]
32                grad_w2 = activation(z)
33                grad_w1 = W2[j] * derivative(z) * x
34                grad_b1 = W2[j] * derivative(z)
35
36                W2[j] -= lr * error * grad_w2
37                W1[j] -= lr * error * grad_w1
38                b1[j] -= lr * error * grad_b1
39
40    max_error = max(abs(forward(x) - target_func(x)) for x in x_vals)
41    assert max_error < 2.0, "Le modele a diverge de la cible
42        oscillatoire."
43    return max_error

```

```

42 x_points = [x/10.0 for x in range(-31, 32)]
43 err = approx_sinus(x_points, lambda x: math.sin(x), 30)
44 print("Erreur maximale de la reconstruction du sinus:", err)

```

20 Simulation de la fonction de Heaviside avec régularisation

Ce TP explore une facette spécifique du théorème d'approximation universelle en implémentant en Python pur (sans aucune dépendance) une preuve de concept algorithmique.

```

1 def heaviside_approximation(x_vals, n_neurons):
2     """
3     Apprentissage d'un saut abrupt. Demontre la limite du modele sur les
4     discontinuités.
5     """
6     import random
7
8     def target(x):
9         return 1.0 if x > 0 else 0.0
10
11     W1 = [random.uniform(-5, 5) for _ in range(n_neurons)]
12     b1 = [random.uniform(-1, 1) for _ in range(n_neurons)]
13     W2 = [random.uniform(-1, 1) for _ in range(n_neurons)]
14
15     def sigmoid(z):
16         # Approximation de exp pour eviter les overflow en python pur
17         if z < -20: return 0.0
18         if z > 20: return 1.0
19         exp_z = 2.71828 ** z
20         return exp_z / (1 + exp_z)
21
22     def forward(x):
23         return sum(W2[j] * sigmoid(W1[j] * x + b1[j]) for j in range(
24             n_neurons))
25
26     lr = 0.05
27     for epoch in range(300):
28         for x in x_vals:
29             y_pred = forward(x)
30             err = y_pred - target(x)
31
32             for j in range(n_neurons):
33                 z = W1[j] * x + b1[j]
34                 s = sigmoid(z)
35                 ds = s * (1 - s)
36
37                 W2[j] -= lr * err * s
38                 W1[j] -= lr * err * W2[j] * ds * x
39                 b1[j] -= lr * err * W2[j] * ds
40
41     # Evaluation
42     err_0 = abs(forward(-0.1) - target(-0.1))
43     err_1 = abs(forward(0.1) - target(0.1))
44     assert err_0 >= 0 and err_1 >= 0
45     return err_0, err_1

```

```

45 pts = [x/10.0 for x in range(-10, 11)]
46 e0, e1 = heaviside_approximation(pts, 15)
47 print("Erreur d'approximation autour du saut:", e0, e1)

```

21 Fonction multivariable 2D (Approximation de XOR continu)

Ce TP explore une facette spécifique du théorème d'approximation universelle en implémentant en Python pur (sans aucune dépendance) une preuve de concept algorithmique.

```

1 def approx_multivar(data_points, n_neurons):
2     """
3     Approximation en dimension superieure (2 vers 1).
4     Demontre la capacite du modele a casser la planarite.
5     """
6     import random
7
8     # 2 variables d'entree
9     W1 = [[random.uniform(-1, 1), random.uniform(-1, 1)] for _ in range(
10         n_neurons)]
11     b1 = [random.uniform(-1, 1) for _ in range(n_neurons)]
12     W2 = [random.uniform(-1, 1) for _ in range(n_neurons)]
13
14     def relu(z):
15         return z if z > 0 else 0
16
17     def forward(x1, x2):
18         s = 0
19         for j in range(n_neurons):
20             z = W1[j][0] * x1 + W1[j][1] * x2 + b1[j]
21             s += W2[j] * relu(z)
22         return s
23
24     lr = 0.02
25     for epoch in range(400):
26         for x1, x2, y_true in data_points:
27             y_pred = forward(x1, x2)
28             err = y_pred - y_true
29
30             for j in range(n_neurons):
31                 z = W1[j][0] * x1 + W1[j][1] * x2 + b1[j]
32                 if z > 0:
33                     W2[j] -= lr * err * relu(z)
34                     W1[j][0] -= lr * err * W2[j] * x1
35                     W1[j][1] -= lr * err * W2[j] * x2
36                     b1[j] -= lr * err * W2[j]
37
38     mse = sum((forward(x1, x2) - y)**2 for x1, x2, y in data_points) /
39     len(data_points)
40     assert mse >= 0.0
41     return mse
42
43 # XOR continu
44 dataset = [
45     (0, 0, 0), (1, 1, 0), (0, 1, 1), (1, 0, 1),
46     (0.5, 0.5, 0.5), (0.2, 0.8, 0.8), (0.9, 0.1, 0.8)

```

```

45 ]
46 mse_final = approx_multivar(dataset, 10)
47 print("MSE multivarié:", mse_final)

```

22 Évaluation comparative des fonctions d'activation

Ce TP explore une facette spécifique du théorème d'approximation universelle en implémentant en Python pur (sans aucune dépendance) une preuve de concept algorithmique.

```

1 def evaluate_activations(x_vals, target_func, n_neurons):
2     """
3     Compare la vitesse de convergence de ReLU et tanh sur la meme cible.
4     """
5     import random
6
7     def train_network(activation_type):
8         W1 = [random.uniform(-0.5, 0.5) for _ in range(n_neurons)]
9         b1 = [0.0 for _ in range(n_neurons)]
10        W2 = [random.uniform(-0.5, 0.5) for _ in range(n_neurons)]
11
12        lr = 0.01
13        for epoch in range(100):
14            for x in x_vals:
15                # Forward
16                zs = [W1[j] * x + b1[j] for j in range(n_neurons)]
17                if activation_type == "relu":
18                    acts = [z if z > 0 else 0 for z in zs]
19                    derivs = [1 if z > 0 else 0 for z in zs]
20                else:
21                    # tanh approche
22                    acts = [(2.718**z - 2.718**(-z))/(2.718**z +
23                        2.718**(-z)) if abs(z)<10 else (1 if z>0 else -1)
24                        for z in zs]
25                    derivs = [1 - a*a for a in acts]
26
27                y_pred = sum(W2[j] * acts[j] for j in range(n_neurons))
28                err = y_pred - target_func(x)
29
30                # Backward
31                for j in range(n_neurons):
32                    W2[j] -= lr * err * acts[j]
33                    W1[j] -= lr * err * W2[j] * derivs[j] * x
34                    b1[j] -= lr * err * W2[j] * derivs[j]
35
36                mse = sum((sum(W2[j] * (W1[j]*x+b1[j] if W1[j]*x+b1[j]>0 else 0)
37                    if activation_type=="relu" else 0) - target_func(x))**2 for
38                    x in x_vals) / len(x_vals)
39            return mse
40
41        mse_relu = train_network("relu")
42        mse_tanh = train_network("tanh")
43        assert mse_relu >= 0 and mse_tanh >= 0
44        return mse_relu, mse_tanh
45
46 pts = [x/5.0 for x in range(-5, 6)]

```

```
43 # Target = x^3
44 m1, m2 = evaluate_activations(pts, lambda x: x*x*x, 15)
45 print("Comparaison terminee.")
```